

Lecture 6: Assembly I — x86-64 Basics, Registers & Data Movement

Session 7 · Mon Sep 21, 2026

Contents

Your Program Was Never C	1
Where We Are	2
Why Anyone Reads This	2
One Line, Four Forms	2
The Code We Read Today	2
Run It: Get the Assembly Yourself	3
Where the Registers Actually Are	3
Sixteen Boxes	4
So What Is <code>movq</code> ? Not a Box.	4
One Register, Four Names	5
Run It: Writing Through the Narrow Names	5
Two Syntaxes for the Same Bytes	5
<code>mov</code> Is a Copy	5
The Parentheses Mean “Go Get It”	6
Widening: Is <code>0xFF 255</code> , or <code>-1</code> ?	6
Addressing Modes	7
Run It: One Line of C, One Addressing Mode	7
<code>lea</code> Computes, It Does Not Load	8
Run It: <code>lea</code> as the Cheap Multiplier	8
Your First Whole Function	8
Arithmetic and Logic	8
The <code>FLAGS</code> Register	9
<code>cmp</code> and <code>test</code> Throw the Answer Away	9
A Loop, Line by Line	9
Run It: The Bytes Are Right There	10
The Four Shapes, in a Register	10
AI Systems Connection	10
The Lecture in One Slide	10
Practice	11
Reading	11

Your Program Was Never C

The processor has never seen a `for` loop, a `struct`, or a variable name. By the time your program runs, all of that is gone. What is left is a list of very small instructions over sixteen boxes:

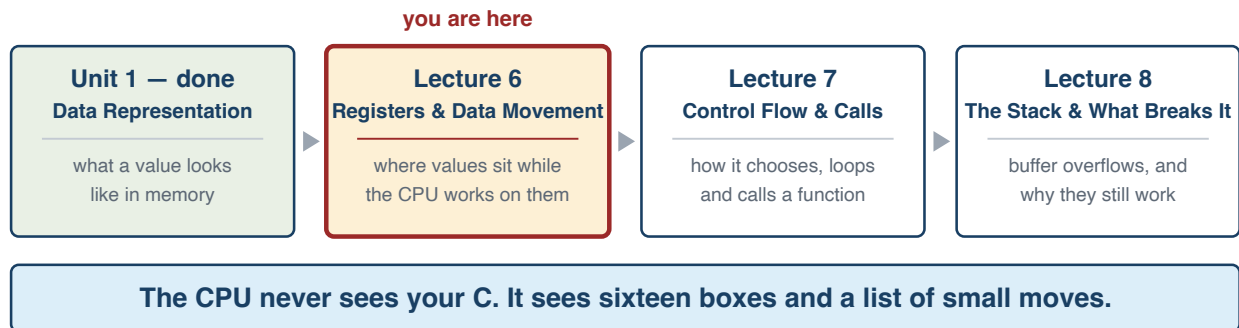
```
total += arr[i];           // what you wrote
```

```
addq (%rdi), %rax          # what runs
```

Unit 1 asked what a **value** looks like in memory. Unit 2 asks what a **program** looks like. Same question, other half of the machine.

Where We Are

Unit 1 asked what a value is. Unit 2 asks what a program is.



Three lectures. Today is the vocabulary: where values sit, and how they move. Nothing today branches or calls anything — that is Wednesday.

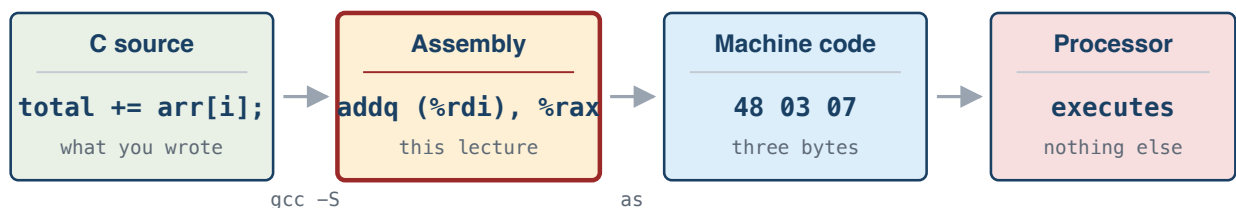
Why Anyone Reads This

You will not write assembly for a living. You will read it — when the crash report has no source, when the profiler blames a line that looks free, or when the optimizer deletes a check and you want to know why.

It is also the only honest answer to “*is this actually faster?*” The source code is a proposal. The assembly is what the machine agreed to.

One Line, Four Forms

The processor has never seen your C



The middle box is the last form a human can still read — and it is one-to-one with the bytes.

Read the assembly and you are reading what actually runs, not what you hoped you wrote.

Assembly is not a summary of the machine code — it is the machine code, spelled out. One instruction, one line, and back again. That is what makes it worth reading.

The Code We Read Today

All of `01-first-look.c` except its `main`. Four functions, and each one comes back later in the lecture:

```

int add(int a, int b)           { return a + b; }
long triple(long x)             { return x * 3; }
long index_it(const long *arr, long i) { return arr[i]; }

long sum(const long *arr, long n) {

```

```

long total = 0;
for (long i = 0; i < n; i++) total += arr[i];
return total;
}

```

Nothing here is clever. That is the point — you already know exactly what each one does, so anything surprising in the assembly is the *machine*, not the algorithm.

Run It: Get the Assembly Yourself

No debugger needed. Ask the compiler for assembly instead of a program — `-S` means “stop after assembly”:

```
cc -O1 -S 01-first-look.c -o 01-first-look.s
```

The first thing in `01-first-look.s`:

```

_add:
    leal    (%rdi,%rsi), %eax
    retq

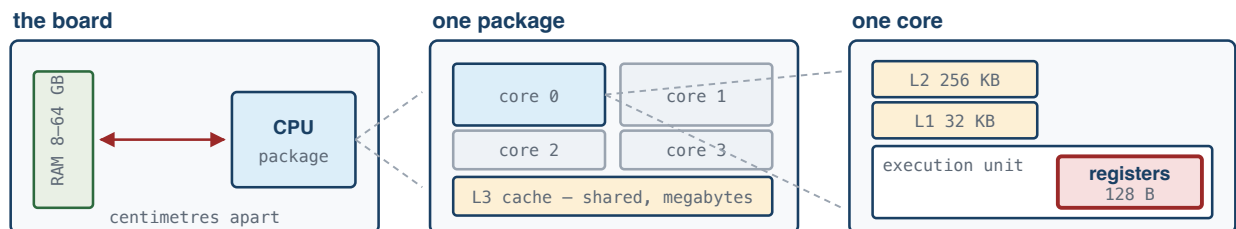
```

That is `add`, whole. Two instructions, and neither one is `add`.

On Apple Silicon `add -arch x86_64; make asm in sample code/lecture06/` does it for you.

Where the Registers Actually Are

Registers are not near the CPU. They are *inside* it.



A cache is **addressed**; a register is **named**. `movq %rbx, %rax` says **rax** — the ISA has no word for “L1”.

Delete every cache and your program still runs, slower. Delete the registers and there is no instruction set.

How much slower each step is — that is Lecture 15.

Lecture 2 opened with a photograph of a RAM module — memory you can hold. The registers are not on that stick, and they are **not the cache** either: they are inside the execution unit, on the same silicon that does the arithmetic.

Sixteen Boxes

Sixteen boxes, and the ABI says what each one is for

Arguments, in this exact order

— the 7th onward go on the stack



Result, and free scratch

— a call may clobber these, so save them first if you care



Callee-saved

— if you use one, you must put it back before you return



Not yours to choose



None of this is enforced by the hardware. It is a convention — and everyone keeps it, or nothing links.

Registers are the only storage the CPU computes on directly; everything else — **cache included** — is a trip to memory. Sixteen is not many, which is why most of what a compiler does is decide who gets one.

Learn the top row first — `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` — the order arguments arrive in.

So What Is `movq`? Not a Box.

Sixteen registers, and **none of them is called `mov`**. Every line has the same two parts:

```
movq    %rbx, %rax
```

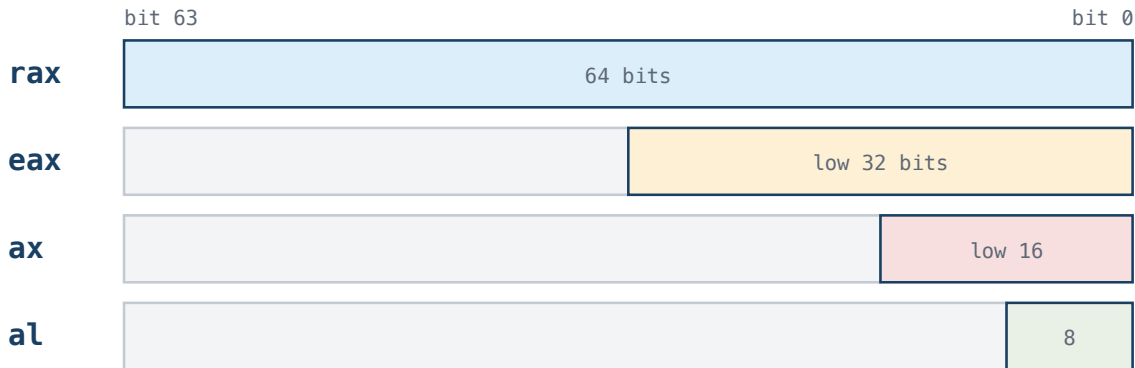
- **`movq`** is the **verb** — the operation. `movq`, `addq`, `leaq`, `jne`. There are a few hundred of these; you will meet about fifteen today.
- **`%rbx`, `%rax`** are the **operands** — what it works on. Each is a register, a constant (`$42`), or a place in memory (`(%rbx)`).

The last letter of the verb is the **size**: `q` = 8 bytes, `l` = 4, `w` = 2, `b` = 1. So `movq` moves eight bytes and `movb` moves one.

Every line you read today is a verb, then the boxes it acts on.

One Register, Four Names

One register, four names — not four registers



Careful: writing **eax** zeroes the top 32 bits. Writing **ax** or **al** leaves them untouched.

rax and **eax** are not two registers. They are two ways of naming the same bits — which is why an `int` (32-bit) and a `long` (64-bit) in the same function keep showing up as **eax** and **rax** in alternating lines.

The rule that catches people: writing **eax** zeroes the top half. Writing **ax** or **al** does not.

Run It: Writing Through the Narrow Names

```
# start with    %rax = 0xAAAAAAAABBBBBBB
# then write 0x11 through each name:

movb $0x11, %al    # 0xAAAAAAAABBBBBB11
movw $0x11, %ax    # 0xAAAAAAAABBBB0011
movl $0x11, %eax   # 0x0000000000000011  <- top half wiped
```

%al and **%ax** leave the rest alone. **%eax** wipes it. That asymmetry is a hardware decision, not a C one — and it is why the compiler writes `xorl %eax, %eax` when it wants a zeroed **%rax**: two bytes instead of seven.

Two Syntaxes for the Same Bytes

The same instruction is written two ways, and you will meet both:

AT&T — we use this	Intel — Intel's manuals, online docs
<code>movq %rax, %rbx</code>	<code>mov rbx, rax</code>
<code>addq \$8, %rsp</code>	<code>add rsp, 8</code>
<code>movq 8(%rbx), %rax</code>	<code>mov rax, [rbx + 8]</code>

We use AT&T — what `CS:APP` prints and what `gcc`, `gdb` and `objdump` give you by default. **Source first:** destination on the right, `%` on registers, `$` on constants, size as a suffix.

Intel reverses the operands. No `%` signs means you are reading Intel.

mov Is a Copy

```
movq %rbx, %rax    # rax = rbx      register -> register
movq (%rbx), %rax  # rax = *rbx     memory -> register
movq %rax, (%rbx)  # *rbx = rax     register -> memory
```

```
movq $42, %rax    # rax = 42    constant -> register
```

Source first, destination last — it reads left to right, like an arrow. `movq %rbx, %rax` overwrites `rax` and leaves `rbx` alone: it copies, despite the name.

One combination is missing on purpose: there is no `movq (%rax), (%rbx)`. Memory to memory takes two instructions, because it takes two trips.

The Parentheses Mean “Go Get It”

The parentheses mean “go get it”

`movq %rbx, %rax`

the number itself



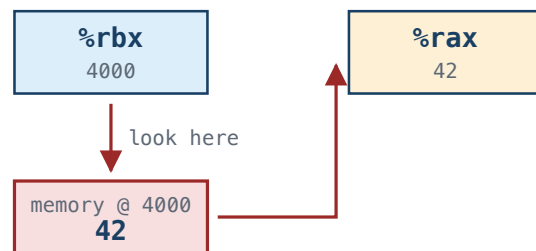
Register to register.

Memory is never touched.

Same instruction. Parentheses change everything.

`movq (%rbx), %rax`

the number stored **at** that address



Two steps: read the address out of `%rbx`, then read the memory it points at.

Lecture 3's pointer dereference, written out: `%rbx` is the pointer, `(%rbx)` is `*rbx`.

Widening: Is `0xFF 255`, or `-1`?

Because the verb carries the size, writing a constant into memory is never ambiguous — `movq $42, (%rbx)` writes eight bytes, `movb $42, (%rbx)` writes one.

Going the other way, from a narrow value to a wide one, needs a **decision**:

```
movzbq %al, %rax    0x00000000000000FF    zero-extend: 255
movsbq %al, %rax    0xFFFFFFFFFFFFFFFF    sign-extend: -1
```

Same byte. Lecture 2's sign extension, now with its own instruction.

Addressing Modes

One address form covers almost every array and struct access

Disp(	Base,	Index,	Scale)
a constant fixed offset	a register where it starts	a register which element	1, 2, 4 or 8 size of one element

All four at once — an array of 8-byte structs:

```
struct P { int x, y; } *a; a[i].y          movl 4(%rdi,%rsi,8), %eax
```

$4 = \text{Disp} \cdot \%rdi = \text{Base} \cdot \%rsi = \text{Index} \cdot 8 = \text{Scale} \cdot y$ sits 4 bytes into each struct

Leave a part out and it takes its default:

<code>(%rdi,%rsi,8)</code>	<code>= %rdi + %rsi*8</code>	no Disp — it is 0
<code>8(%rdi)</code>	<code>= %rdi + 8</code>	no Index, no Scale
<code>(%rdi,%rsi)</code>	<code>= %rdi + %rsi</code>	no Scale — it is 1
<code>(%rdi)</code>	<code>= %rdi</code>	just *p — a plain dereference

An omitted Disp is 0. An omitted Scale is 1. Nothing is ever left undefined.

Base + Index × Scale + Disp, where Scale is 1, 2, 4 or 8 — the sizes things actually are.

Parts you leave out are not missing, they are defaulted: no Disp means 0, no Scale means 1. That is why `(%rdi,%rsi,8)` is a complete address even though the form has four slots and you can only see three.

Run It: One Line of C, One Addressing Mode

```
long index_it(const long *arr, long i) {
    return arr[i];
}
```

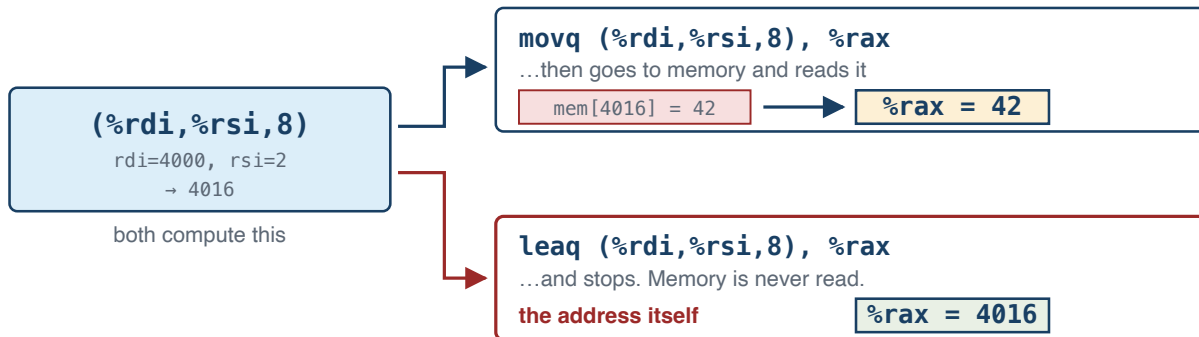
```
_index_it:
    movq    (%rdi,%rsi,8), %rax
    retq
```

`arr` arrived in `%rdi`, `i` in `%rsi`, the answer leaves in `%rax`, and a `long` is 8 bytes so the scale is 8. The entire function body is one addressing mode.

Read `(%rdi,%rsi,8)` as *base*, *index*, *scale* — exactly the order CS:APP writes it in.

leaq Computes, It Does Not Load

One address expression. Two instructions. Two answers.



So **leaq** is used as plain arithmetic. A different example:

`leaq (%rdi,%rdi,2), %rax` computes `rax = rdi * 3` — one instruction, no memory, no multiplier.

leaq — **L**oad **E**ffective **A**ddress. Despite the name, it usually has nothing to do with addresses: the compiler reaches for it in code containing no pointers at all, because it is the one instruction that does multiply-and-add for free.

Run It: leaq as the Cheap Multiplier

```
long times3(long x) { return x * 3; }
long times7(long x) { return x * 7; }
```

```
_times2: leaq (%rdi,%rdi), %rax    # x*2
_times3: leaq (%rdi,%rdi,2), %rax # x*3
_times5: leaq (%rdi,%rdi,4), %rax # x*5
_times9: leaq (%rdi,%rdi,8), %rax # x*9
_affine: leaq 7(,%rdi,8), %rax    # 8x + 7, one instruction

_times7: leaq (,%rdi,8), %rax     # 7 is not Base + Index*Scale,
      subq %rdi, %rax            # but 8x - x is
```

Not one `imul` anywhere. When you see `leaq` in compiler output, read it as arithmetic first and an address second.

Your First Whole Function

The two lines from the start of the lecture:

```
int add(int a, int b) { return a + b; }
```

```
_add:
    leal    (%rdi,%rsi), %eax
    retq
```

Read it off the conventions: first argument in `rdi`, second in `rsi`, return value in `rax` — `eax` here because the result is an `int`. And the addition is done by `leaq`, not `add`, because that way the answer lands straight in `eax` without disturbing either input.

Nothing else happens. No stack, no setup. This is what “the compiler is good at this” looks like.

Arithmetic and Logic

Instruction	Meaning
<code>add dst, src</code>	<code>dst += src</code>
<code>sub dst, src</code>	<code>dst -= src</code>
<code>inc / dec dst</code>	<code>dst++ / dst--</code>
<code>neg dst</code>	<code>dst = -dst</code>
<code>imul dst, src</code>	signed multiply
<code>and / or / xor dst, src</code>	the Lecture 2 bitwise operators
<code>not dst</code>	flip every bit
<code>shl dst, n</code>	<code>dst <<= n</code>
<code>shr dst, n</code>	<code>dst >>= n</code> , filling with zeros
<code>sar dst, n</code>	<code>dst >>= n</code> , filling with the sign bit

Two shifts right, exactly as in Lecture 2: `shr` for unsigned, `sar` for signed. The **type in your C** is what picks the instruction — and it is the only thing that does.

The FLAGS Register

Every arithmetic instruction quietly writes down four facts about its result:

Flag	Set when
ZF	the result was zero
SF	the result was negative (top bit set)
CF	it carried out — unsigned overflow
OF	it overflowed — signed overflow

Lecture 5 said unsigned and signed overflow are different questions. Here they are, as two different bits, recorded at the same time by the same instruction. Nothing decides which one mattered until something reads them.

`cmp` and `test` Throw the Answer Away

There is no “compare” circuit. `cmp` is a subtraction that discards its result and keeps only the flags:

```
cmpq %rbx, %rax    # compute rax - rbx, set flags, keep nothing
testq %rax, %rax    # compute rax & rax, set flags, keep nothing
```

Warning

Read `cmp` carefully: `cmpq %rbx, %rax` computes `rax - rbx` — the *second* operand minus the first. Source-first bites hardest here, because the subtraction reads backwards from the line.

So it sets ZF exactly when `rax == rbx`, and `testq %rax, %rax` sets ZF exactly when `rax` is zero — the cheapest way to ask “is this zero or NULL?”.

The flags sit there until something acts on them. That is Wednesday.

A Loop, Line by Line

```
long total = 0;                                // arr in rdi, n in rsi
for (long i = 0; i < n; i++) total += arr[i];
return total;
```

```
_sum:
    testq %rsi, %rsi        # n <= 0?
    jle  .empty             # if so, skip the loop entirely
    xorl  %ecx, %ecx        # i = 0
```

```

    xorl  %eax, %eax           # total = 0
.loop:
    addq  (%rdi,%rcx,8), %rax   # total += arr[i]
    incq  %rcx                 # i++
    cmpq  %rcx, %rsi           # compares rsi - rcx, i.e. n vs i
    jne   .loop                # if not equal, go round again
    retq

```

Every piece is from today; only the jumps are new. And note the compiler checked `n <= 0` **before** the loop — your `for` became a guard plus a do-while.

Run It: The Bytes Are Right There

```
objdump -d 01-first-look.o
```

```

0000000000000000 <_add>:
    0: 8d 04 37          leal  (%rdi,%rsi), %eax
    3: c3              retq

0000000000000020 <_index_it>:
   20: 48 8b 04 f7        movq  (%rdi,%rsi,8), %rax
   24: c3              retq

```

Three bytes for the `leal`, four for the `mov`, one for `ret`. The `48` in front is the prefix that says “make this one 64-bit” — which is literally why the same instruction on `rax` is one byte longer than on `eax`.

The Four Shapes, in a Register

Lecture 5b ended by promising you would recognise its four shapes in a register dump. A register is 8 bytes; here they are:

int 42	4 bytes	0x000000000000002A
long -1	8 bytes	0xFFFFFFFFFFFFFFFF
int * (an address)	8 bytes	0x0000000016F8467B8
float 3.0f	4 bytes	0x0000000040400000
double 3.0	8 bytes	0x4008000000000000
struct point{1,2}	8 bytes	0x0000000200000001

`0x40400000` is the `3.0f` you decoded on Tuesday. `{1,2}` is `y` then `x`, little-endian, with the alignment from Lecture 4.

Nothing in the register says which shape it holds. The instruction that touches it decides — `add` reads it as an integer, `addss` as a float, `[...]` as an address. That is the whole unit in one sentence, and now you can see it.

AI Systems Connection

The kernels that dominate a training profile — `matmul`, `attention`, `softmax` — are hand-tuned, and the tuning is done by reading exactly what you read today. A register spill in an inner loop shows up as an unexpected `mov` to `[rsp + N]`. That is how you find it.

The other half is security. The ML stack is full of C parsers — `ONNX`, `safetensors`, image decoders — and a malicious model file need not break the math. It can corrupt the program instead.

Lecture 8 is that attack in full. Today you learned to read the ammunition.

The Lecture in One Slide

	what it is	the thing to remember
registers	16 boxes the CPU computes in	<code>rdi</code> , <code>rsi</code> , <code>rdx</code> , <code>rcx</code> , <code>r8</code> , <code>r9</code> are the arguments, in order
<code>rax</code> / <code>eax</code>	one register, two widths	writing <code>eax</code> zeroes the top half
<code>mov</code>	a copy	destination first; no memory-to-memory
<code>[...]</code>	go to that address	it is Lecture 3's <code>*</code> , spelled differently
<code>lea</code>	the same arithmetic, no memory	it is a multiplier, not a pointer thing
flags	facts about the last result	<code>cmp</code> and <code>test</code> set them and keep nothing else

Next: **Control Flow & Calls** — the jumps that read those flags, and what a function call actually costs.

Practice

1. Write a one-line C function and get its assembly with `cc -O1 -S`. Identify which register each argument arrived in.
2. What is the difference between `movq %rbx, %rax`, `movq (%rbx), %rax`, and `leaq (%rbx), %rax`? Give the value in `%rax` for each, if `%rbx` holds 4000 and memory at 4000 holds 42.
3. `leaq (%rdi,%rdi,4), %rax` — what does this compute? Write the C.
4. Why can the compiler use `lea` for `x * 5` but not for `x * 7` in one instruction? What does it do instead?
5. A function starts with `testq %rdi, %rdi`. What is the C almost certainly checking?
6. Write `long get(struct s *p)` returning a field 16 bytes into the struct. Predict the addressing mode before you compile it, then check.
7. Take `sum` from the slides and change `long` to `int`. Which registers change name, and why does the addressing mode's scale change?

Reading

- CS:APP §3.1–3.3 (a historical perspective, encodings, data formats)
- CS:APP §3.4 (accessing information — registers and operand forms)
- CS:APP §3.5 (arithmetic and logical operations)

Note that CS:APP is written in **AT&T** syntax; these slides are in Intel. Operand order is reversed between them.